Istanbul Medipol University
School of Engineering and Natural Sciences
Computer Engineering

Fundamentals of Programming Languages
Fall 2026 - Mini Project

Building a Compiler with Lex & Yacc

Report

Student name	Kenan Saleh
Student number	20220702145
Course	Fundamentals of Programming Languages
Date	24.10.2026

Contents

1. Cover page
2. Environment
3. How lex and yacc work
4. Grammar file explanations
5. How I ran the examples
6. Sample runs (T1 to T6 and bonus)
7. Experiments (T7)
8. Answers to the key questions
9. Conclusion
10. References

All source files are in `src/` and all inputs and saved outputs are in `inputs/` and `outputs/` inside the zip. Every listing of a run in this report is copied from the files in `outputs/`.

2. Environment

I did the project on a Linux virtual machine. The table below shows the setup I used.

Table 1: *Environment used for the project*

Item	Value
Host computer	[fill in: your laptop / OS]
Virtualisation software	[fill in: e.g. Oracle VirtualBox 7.x]
VM resources	2 CPU cores, 4 GB RAM, 25 GB disk
Linux distribution	Ubuntu 24.04 LTS (64 bit) desktop
Kernel (uname -r)	[fill in from uname -a]
flex	flex 2.6.4
bison	bison (GNU Bison) 3.8.2
gcc	gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
make	GNU Make 4.3

I installed the tools with the commands from the project sheet:

```
sudo apt update
sudo apt install -y flex bison gcc make
flex --version
bison --version
gcc --version
uname -a
```

[Paste screenshot here: VM settings window showing 2 CPUs / 4 GB RAM / 25 GB disk]

Figure 1: *VirtualBox settings of the VM (CPU, RAM, disk) for Part 1*

[Paste screenshot here: Ubuntu desktop inside the VM]

Figure 2: *Installed Ubuntu desktop running inside the VM*

[Paste screenshot here: terminal with the four version commands]

Figure 3: *Output of flex --version, bison --version, gcc --version and uname -a*

3. How lex and yacc work

A compiler does not read a program as one big string. First it cuts the text into small pieces called tokens and after that it checks if the order of the tokens follows the grammar of the language. The first step is lexical analysis and the second step is parsing. Lex (on Linux it is flex) writes the lexical analyzer for us and yacc (on Linux it is bison) writes the parser.

Lex. In a `.l` file I write regular expressions and next to each one a small piece of C code (the action). flex turns this file into `lex.yy.c` which contains the function `yylex()`. Every time `yylex()` is called it reads characters from `yyin` (stdin by default) and finds the longest text that matches one of the patterns. If two patterns match the same length the one written first wins. The matched text is in `yytext` and its length is in `yyleng`. Then the action runs. If the action does `return` then `yylex()` gives that token back to whoever called it. If nothing matches, lex just copies the character to the output (this is the default `ECHO`). When the input ends lex calls `yywrap()`; returning 1 means there is no more input.

A `.l` file has three parts separated by `%%`: the definitions part (C code inside `%{ %}` and named patterns like `digit [0-9]`), the rules part (pattern + action) and the user subroutines part (normal C functions such as `main` or `yywrap`).

Yacc. In a `.y` file I write the grammar in BNF form. Each rule has a left side, a colon and the right side, and it can have a C action in braces. bison turns this into `y.tab.c` which contains `yyparse()`. With the `-d` option it also writes `y.tab.h` with the token numbers. The parser is a shift/reduce (LALR(1)) parser. It keeps a stack. When it reads a token it can **shift** it onto the stack, or when the top of the stack matches the right side of a rule it can **reduce** it to the left side. During a reduce the action runs: `$1`, `$2`, ... are the values of the symbols on the right side and `$$` is the value of the new left side symbol. A `.y` file also has three parts: declarations (`%token`, `%left`, `%union`, ...), rules and user code.

How they work together. `yyparse()` calls `yylex()` every time it needs a new token. `yylex()` returns the token **type** (a number) and puts the token **value** into the global variable `yylval`. Single characters like `'+'` are returned as their own ASCII code. Named tokens like `INTEGER` get numbers starting from 258 and these numbers are in `y.tab.h`. The lex file includes `y.tab.h` so both sides use the same numbers. In the end gcc compiles `y.tab.c` and `lex.yy.c` and links them with the other C files. For the calculator the same parser can be linked with three different back ends: an interpreter, a compiler and a tree drawer. Figure 4 shows this pipeline.

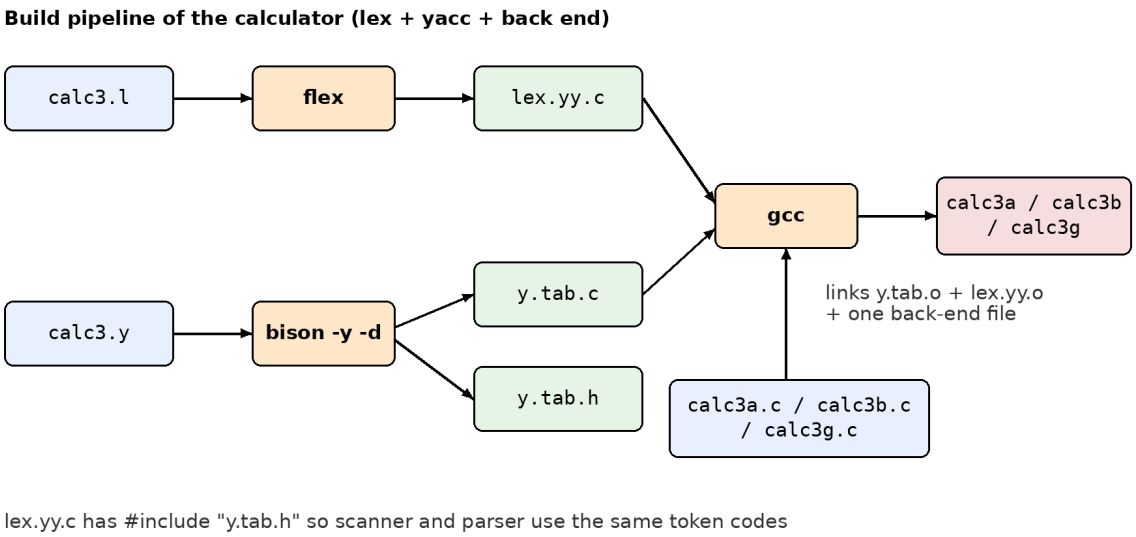


Figure 4: *Build pipeline: .l to flex to lex.yy.c, .y to bison to y.tab.c and y.tab.h, then gcc to the executable*

So the short version is: flex makes the scanner, bison makes the parser, the parser asks the scanner for tokens one by one, and the actions inside the grammar compute values or build a syntax tree.

4. Grammar file explanations

In this part I explain every `.l` and `.y` file I used in T1 to T6 (and the bonus version). I removed the comments from the source files so I explain every part here instead.

4.1 Lex-only examples (T1)

These four programs are from the Lex section of the tutorial. They only use lex, there is no parser. For T1 I also made `ex3mod.l` where I changed one rule.

Listing 1: *src/t1/ex1.l: copy the input to the output*

```
%%
.       ECHO;
\n      ECHO;
%%
int yywrap(void) {
    return 1;
}

int main(void) {
    yylex();
    return 0;
}
```

ex1.l. The definitions part is empty. In the rules part `.` matches any one character except newline and `\n` matches the newline. Both actions are `ECHO` which prints `yytext`. So the program just copies its input. The user code part has `yywrap()` returning 1 (no more files) and `main()` which calls `yylex()` once. `yylex()` returns only at the end of file because no action has a return.

Listing 2: *src/t1/ex2.l: print every line with a line number*

```
%{
    int lineno;
%}
%%
^(.*)\n     printf("%4d\t%s", ++lineno, yytext);
%%
int main(int argc, char *argv[]) {
    yyin = fopen(argv[1], "r");
    yylex();
    fclose(yyin);
```

```
    return 0;
}
```

ex2.l. The definitions part declares a counter. The only rule `^(.*)\n` matches a whole line: `^` means the start of a line, `(.*)` is any characters and `\n` is the end of the line. The action increases the counter and prints it with the line. `main()` opens the file given as the first argument and puts it in `yyin`, so lex reads from the file and not from stdin. In the tutorial the counter is called `yylineno` but I had to rename it (see Section 5.3).

Listing 3: *src/t1/ex3.l: count identifiers*

```
digit       [0-9]
letter      [A-Za-z]
%{
    int count;
%}
%%
{letter}({letter}|{digit})*     count++;
%%
int main(void) {
    yylex();
    printf("number of identifiers = %d\n", count);
    return 0;
}
```

ex3.l. The definitions part makes two named patterns, `digit` and `letter`, and declares `count`. The rule `{letter}({letter}|{digit})*` means a letter and then any number of letters or digits, which is an identifier. The action only does `count++`. Everything else is not matched, so lex ECHOes it. At the end `main()` prints the number of identifiers.

Listing 4: *src/t1/ex4.l: count characters, words and lines (like wc)*

```
    int nchar, nword, nline;
%%
\n          { nline++; nchar++; }
[^ \t\n]+   { nword++, nchar += yyleng; }
.           { nchar++; }
%%
int main(void) {
    yylex();
    printf("%d\t%d\t%d\n", nchar, nword, nline);
    return 0;
}
```

ex4.l. The line before the first %% starts with spaces so lex copies it to the C file as it is, this declares the three counters. Rule 1 `\n` adds one line and one character. Rule 2 `[^ \t\n]+` is one or more characters that are not a space, tab or newline, which is a word; it adds 1 to `nword` and `yyleng` to `nchar`. Rule 3 `.` is every other single character (spaces and tabs) and adds one character. Because of the longest match rule, rule 2 always wins over rule 3 for normal text. There is no `yywrap()` so I link with `-lfl`.

Listing 5: *The one rule I changed in ex3mod.l (diff against ex3.l)*

```
$ diff ex3.l ex3mod.l
2c2
< letter      [A-Za-z]
---
> letter      [A-Za-z_]
```

ex3mod.l. I only added _ to the `letter` class. Now names like `my_var` or `_tmp` are one identifier. Before the change `my_var` was counted as two identifiers (`my` and `var`) and the _ was echoed. The effect is shown in Section 6.1.

4.2 Practice Part I: calc1 (T2)

Listing 6: *src/t2/calc1.l*

```
%{
    #include <stdlib.h>
    void yyerror(char *);
    #include "y.tab.h"
%}

%%

[0-9]+      {
                yylval = atoi(yytext);
                return INTEGER;
            }

[-+\n]      return *yytext;

[ \t]       ;

.           yyerror("invalid character");

%%

int yywrap(void) {
    return 1;
}
```

Table 2: *Rules of calc1.l*

Regular expression	Action	Returns
`[0-9]+`	converts the digits with atoi and stores the number in yylval	INTEGER (value in yylval)
`[-+\n]`	returns the character itself	'-' or '+' or '\n' (ASCII code)
`[ \t]`	nothing, spaces and tabs are skipped	nothing
`.`	calls yyerror("invalid character")	nothing

The definitions part includes `y.tab.h` so the scanner knows the number of `INTEGER`. In the rule `[-+\n]` the `-` is first so it is not read as a range. Since there is no `%union`, `yylval` is an `int`.

Listing 7: *src/t2/calc1.y*

```
%{
    #include <stdio.h>
    int yylex(void);
    void yyerror(char *);
%}

%token INTEGER

%%

program:
        program expr '\n'         { printf("%d\n", $2); }
        |
        ;

expr:
        INTEGER                   { $$ = $1; }
        | expr '+' expr           { $$ = $1 + $3; }
        | expr '-' expr           { $$ = $1 - $3; }
        ;

%%

void yyerror(char *s) {
    fprintf(stderr, "%s\n", s);
}
```

```
int main(void) {
    yyparse();
    return 0;
}
```

Declarations. `%token INTEGER` makes the token (it gets the number 258 in `y.tab.h`). There are no precedence lines in this version. **Productions** in BNF:

```
<program> ::= <program> <expr> '\n'  |  ε
<expr>    ::= INTEGER  |  <expr> '+' <expr>  |  <expr> '-' <expr>
```

Actions. `program` is left recursive, so it reads many lines. After each line the action prints `$2`, which is the value of the expression. For `expr: INTEGER` the action `$$ = $1` copies the number that lex put in `yylval`. For `+` and `-` the action adds or subtracts the values of the two sub-expressions. The user code part has `yyerror()` (prints the message) and `main()` which calls `yyparse()`.

Trace of an INTEGER token. For `1 + 2`: `yyparse()` calls `yylex()`, lex matches `1` with `[0-9]+`, stores 1 in `yylval` and returns `INTEGER`. The parser shifts `INTEGER` and copies `yylval` to the value stack, then reduces it to `expr` (now `$1` is 1). Next lex returns `'+'`, then `INTEGER` with `yylval` = 2, then `'\n'`. When the parser sees `'\n'` it reduces `expr '+' expr` and the action makes `$$ = 1 + 2 = 3`, and the `program` rule prints 3.

bison gives the warning `4 shift/reduce conflicts` for this file because `expr '+' expr` is ambiguous without `%left`. bison chooses shift, which makes `+` and `-` right associative. I explain the effect in Section 6.2 and Section 8.

4.3 Practice Part II: calc2 (T3)

Listing 8: *src/t3/calc2.l*

```
%{
    #include <stdlib.h>
    #include "y.tab.h"
    void yyerror(char *);
%}

%%

[a-z]       {
                yylval = *yytext - 'a';
                return VARIABLE;
            }

[0-9]+      {
                yylval = atoi(yytext);
                return INTEGER;
            }

[-+()=/*\n] { return *yytext; }

[ \t]       ;

.           yyerror("invalid character");

%%

int yywrap(void) {
    return 1;
}
```

Table 3: *Rules of calc2.l*

Regular expression	Action	Returns
`[a-z]`	yylval = *yytext - 'a' (0 for a, 25	VARIABLE (index in yylval)

	for z)	
`[0-9]+`	yylval = atoi(yytext)	INTEGER (value in yylval)
`[-+()=/*\n]`	returns the character	the character itself
`[ \t]`	skip	nothing
`.`	yyerror("invalid character")	nothing

Listing 9: *src/t3/calc2.y*

```
%{
    #include <stdio.h>
    void yyerror(char *);
    int yylex(void);

    int sym[26];
%}

%token INTEGER VARIABLE
%left '+' '-'
%left '*' '/'

%%

program:
        program statement '\n'
        |
        ;

statement:
        expr                    { printf("%d\n", $1); }
        | VARIABLE '=' expr     { sym[$1] = $3; }
        ;

expr:
        INTEGER
        | VARIABLE              { $$ = sym[$1]; }
        | expr '+' expr         { $$ = $1 + $3; }
        | expr '-' expr         { $$ = $1 - $3; }
        | expr '*' expr         { $$ = $1 * $3; }
        | expr '/' expr         { $$ = $1 / $3; }
        | '(' expr ')'          { $$ = $2; }
        ;

%%

void yyerror(char *s) {
    fprintf(stderr, "%s\n", s);
}

int main(void) {
    yyparse();
    return 0;
}
```

Declarations. `int sym[26]` is the symbol table, one int for each letter. `%token INTEGER VARIABLE` makes two tokens. `%left '+' '-'` and then `%left '*' '/'` give precedence: lines written later have higher precedence, and `%left` means left associative. **Productions** in BNF:

```
<program>   ::= <program> <statement> '\n'  |  ε
<statement> ::= <expr>  |  VARIABLE '=' <expr>
<expr>      ::= INTEGER | VARIABLE | <expr> '+' <expr> | <expr> '-' <expr>
              | <expr> '*' <expr> | <expr> '/' <expr> | '(' <expr> ')'
```

Actions. A statement that is only an expression prints its value. `VARIABLE '=' expr` stores the value in `sym[$1]` where `$1` is the index from lex. `expr: VARIABLE` reads `sym[$1]`. The four operators compute the result directly and `'(' expr ')'` just passes `$2` up. `expr: INTEGER` has no action, yacc uses the default `$$ = $1`. Because `sym` is a global array it starts as zeros, so an undefined variable is 0.

How %left gives * higher precedence than +. When the parser has `expr '+' expr` on the stack and the next token is `'*'`, it can reduce the addition or shift the `*`. bison compares the precedence of the rule (the precedence of `'+'`) with the precedence of the token `'*'`. `'*'` is declared on a later line so it is higher, and bison shifts. So the multiplication is done first. If the next token is `'+'` again the precedences are equal, and `%left` says reduce, so `a - b - c` is `(a - b) - c`.

4.4 The full calculator: calc3 (T4, T5, T6)

Listing 10: *src/calc3/calc3.h: node types of the syntax tree*

```
typedef enum { typeCon, typeId, typeOpr } nodeEnum;

typedef struct {
    int value;
} conNodeType;

typedef struct {
    int i;
} idNodeType;

typedef struct {
    int oper;
    int nops;
    struct nodeTypeTag *op[1];
} oprNodeType;

typedef struct nodeTypeTag {
    nodeEnum type;

    union {
        conNodeType con;
        idNodeType id;
        oprNodeType opr;
    };
} nodeType;

extern int sym[26];
```

calc3.h. There are three kinds of nodes (`nodeEnum`): a constant (`typeCon`) that keeps the integer value, an identifier (`typeId`) that keeps the index into `sym`, and an operator (`typeOpr`). An operator node keeps the operator code (`oper`), the number of operands (`nops`) and an array of pointers to the child nodes. The array is declared with size 1 but `opr()` allocates more memory so it can have more children. `nodeType` has the type and a union of the three, so one node uses only the part it needs. `sym` is declared extern because it is defined in `calc3.y` but used by the back ends.

Listing 11: *src/calc3/calc3.l*

```
%{
#include <stdlib.h>
#include "calc3.h"
#include "y.tab.h"
void yyerror(char *);
%}

%%

[a-z]       {
                yylval.sIndex = *yytext - 'a';
                return VARIABLE;
            }

0           {
                yylval.iValue = atoi(yytext);
                return INTEGER;
            }

[1-9][0-9]* {
                yylval.iValue = atoi(yytext);
```

```
                return INTEGER;
            }

[-()<>=+*/;{}.] {
                return *yytext;
            }

">="        return GE;
"<="        return LE;
"=="        return EQ;
"!="        return NE;
"while"     return WHILE;
"if"        return IF;
"else"      return ELSE;
"print"     return PRINT;

[ \t\n]+    ;

.           yyerror("Unknown character");

%%

int yywrap(void) {
    return 1;
}
```

Table 4: *Rules of calc3.l*

Regular expression	Action / value	Returns
`[a-z]`	yylval.sIndex = index of the letter	VARIABLE
`0`	yylval.iValue = 0	INTEGER
`[1-9][0-9]*`	yylval.iValue = atoi(yytext)	INTEGER
`[-()<>=+*/;{}.]`	-	the character itself
`">=" "<=" "==" "!="`	-	GE, LE, EQ, NE
`"while" "if" "else" "print"`	-	WHILE, IF, ELSE, PRINT
`[ \t\n]+`	skip whitespace and newlines	nothing
`.`	yyerror("Unknown character")	nothing

Now `yylval` is a union so the actions say which field they fill. Numbers are split in two rules so a number cannot start with 0 (except the number 0 itself). For `>=` the longest match rule makes lex choose `GE` and not `'>'` followed by `'='`. For a word like `while` the pattern `[a-z]` only matches one letter, so the keyword rule (5 characters) is longer and wins. Newlines are now skipped because statements end with `;`.

Listing 12: *src/calc3/calc3.y: declarations and rules (the helper functions con, id, opr, freeNode, yyerror and main come after the second %% and are explained below)*

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include "calc3.h"

nodeType *opr(int oper, int nops, ...);
nodeType *id(int i);
nodeType *con(int value);
void freeNode(nodeType *p);
int ex(nodeType *p);
int yylex(void);

void yyerror(char *s);
int sym[26];
%}

%union {
    int iValue;
    char sIndex;
```

```
    nodeType *nPtr;
};

%token <iValue> INTEGER
%token <sIndex> VARIABLE
%token WHILE IF PRINT
%nonassoc IFX
%nonassoc ELSE

%left GE LE EQ NE '>' '<'
%left '+' '-'
%left '*' '/'
%nonassoc UMINUS

%type <nPtr> stmt expr stmt_list

%%

program:
        function                { exit(0); }
        ;

function:
          function stmt         { ex($2); freeNode($2); }
        |
        ;

stmt:
          ';'                           { $$ = opr(';', 2, NULL, NULL); }
        | expr ';'                      { $$ = $1; }
        | PRINT expr ';'                { $$ = opr(PRINT, 1, $2); }
        | VARIABLE '=' expr ';'         { $$ = opr('=', 2, id($1), $3); }
        | WHILE '(' expr ')' stmt       { $$ = opr(WHILE, 2, $3, $5); }
        | IF '(' expr ')' stmt %prec IFX { $$ = opr(IF, 2, $3, $5); }
        | IF '(' expr ')' stmt ELSE stmt { $$ = opr(IF, 3, $3, $5, $7); }
        | '{' stmt_list '}'             { $$ = $2; }
        ;

stmt_list:
          stmt                  { $$ = $1; }
        | stmt_list stmt        { $$ = opr(';', 2, $1, $2); }
        ;

expr:
          INTEGER               { $$ = con($1); }
        | VARIABLE              { $$ = id($1); }
        | '-' expr %prec UMINUS { $$ = opr(UMINUS, 1, $2); }
        | expr '+' expr         { $$ = opr('+', 2, $1, $3); }
        | expr '-' expr         { $$ = opr('-', 2, $1, $3); }
        | expr '*' expr         { $$ = opr('*', 2, $1, $3); }
        | expr '/' expr         { $$ = opr('/', 2, $1, $3); }
        | expr '<' expr         { $$ = opr('<', 2, $1, $3); }
        | expr '>' expr         { $$ = opr('>', 2, $1, $3); }
        | expr GE expr          { $$ = opr(GE, 2, $1, $3); }
        | expr LE expr          { $$ = opr(LE, 2, $1, $3); }
        | expr NE expr          { $$ = opr(NE, 2, $1, $3); }
        | expr EQ expr          { $$ = opr(EQ, 2, $1, $3); }
        | '(' expr ')'          { $$ = $2; }
        ;

%%
...
```

Declarations. `%union` says that a value can be an `int` (`iValue`), a `char` index (`sIndex`) or a pointer to a node (`nPtr`). `%token <iValue> INTEGER` and `%token <sIndex> VARIABLE` tell which field these tokens use, and `%type <nPtr> stmt expr stmt_list` says the nonterminals are tree pointers. `WHILE IF PRINT` are plain tokens. `%nonassoc IFX` and `%nonassoc ELSE` are only for the dangling else. Then come the operators from lowest to highest precedence:

comparisons, `+ -`, `* /` and `UMINUS`. `UMINUS` is not a real token, it is only a name used with `%prec` to give unary minus the highest precedence.

Productions in BNF:

```
<program>   ::= <function>
<function>  ::= <function> <stmt>  |  ε
<stmt>      ::= ';' | <expr> ';' | PRINT <expr> ';' | VARIABLE '=' <expr> ';'
              | WHILE '(' <expr> ')' <stmt>
              | IF '(' <expr> ')' <stmt>
              | IF '(' <expr> ')' <stmt> ELSE <stmt>
              | '{' <stmt_list> '}'
<stmt_list> ::= <stmt>  |  <stmt_list> <stmt>
<expr>      ::= INTEGER | VARIABLE | '-' <expr> | '(' <expr> ')'
              | <expr> <op> <expr>
<op>        ::= '+' | '-' | '*' | '/' | '<' | '>' | GE | LE | NE | EQ
```

Table 5: *Semantic actions of calc3.y*

Production	What the action builds or does
`program: function`	exit(0) when all input is parsed
`function: function stmt`	runs the back end on the statement with ex($2) and then frees the tree
`stmt: ';'`	an empty ';' node with two NULL children
`stmt: expr ';'`	the expression tree itself
`stmt: PRINT expr ';'`	opr(PRINT, 1, expr)
`stmt: VARIABLE '=' expr ';'`	opr('=', 2, id(var), expr)
`stmt: WHILE (...) stmt`	opr(WHILE, 2, condition, body)
`stmt: IF (...) stmt`	opr(IF, 2, condition, then-part)
`stmt: IF (...) stmt ELSE stmt`	opr(IF, 3, condition, then-part, else-part)
`stmt: '{' stmt_list '}'`	the tree of the list
`stmt_list: stmt_list stmt`	opr(';', 2, list, stmt) so a block becomes a chain of ';' nodes
`expr: INTEGER / VARIABLE`	con(value) / id(index), the leaves
`expr: '-' expr`	opr(UMINUS, 1, expr)
`expr: expr op expr`	opr(op, 2, left, right)
`expr: '(' expr ')'`	just $2, brackets do not make a node

The user code part has `con()`, `id()` and `opr()` which `malloc` a node and fill it. `opr()` uses `stdarg.h` because it gets a different number of children each time. `freeNode()` frees a tree recursively. The function `ex()` is not here. It comes from the back end file that is linked, so the same parser can run, compile or draw the tree.

How a statement becomes a tree. Take `x = 2 * (3 + 4);`. lex returns `VARIABLE(23) '=' INTEGER(2) '*' '(' INTEGER(3) '+' INTEGER(4) ')' ';'`. The parser reduces the integers to `con(2)`, `con(3)` and `con(4)`. Inside the brackets `expr '+' expr` becomes `opr('+', 2, con(3), con(4))` and the bracket rule passes it up. Then `expr '*' expr` becomes `opr('*', 2, con(2), +node)`. Finally the assignment rule makes `opr('=', 2, id(23), *node)`. When `function: function stmt` is reduced, `ex()` gets the root. The real drawing of this tree is in Section 6.6.

Back ends. `calc3a.c` is an interpreter: `ex()` walks the tree recursively and returns a value, for `WHILE` it uses a real C while loop and for `IF` a real C if. `calc3b.c` is a compiler for a stack machine: for each node it first generates code for the children and then prints one instruction (`push`, `pop`, `add`, `compLT`, `jz`, `jmp`, `print`, ...) and it makes labels `L000`, `L001` for loops and ifs. `calc3g.c` (from Frank Thomas Braun, included in the tutorial) draws the tree as ASCII art in a 200x200 character array: `exNode()` computes the width of each sub-tree and puts the parent in the middle above its children with lines between them.

4.5 Bonus: calc4

For the bonus tasks I copied calc3 to `src/bonus/` and named it calc4. I did all four bonus items. Only the changed parts are shown here, the full files are in the zip.

Listing 13: *Changed parts of src/bonus/calc4.l*

```
%option yylineno
...
[-()<>=+*/%^;{}.] {
                return *yytext;
            }
...
"do"        return DO;
"for"       return FOR;
...
[a-zA-Z][a-zA-Z0-9_]* {
                yylval.sIndex = lookup(yytext);
                return VARIABLE;
            }
```

Listing 14: *Changed parts of src/bonus/calc4.y*

```
%union {
    int iValue;
    int sIndex;
    nodeType *nPtr;
};
...
%token WHILE DO FOR IF PRINT
...
%left '*' '/' '%'
%nonassoc UMINUS
%right '^'

%type <nPtr> stmt expr stmt_list assign
...
        | assign ';'                                    { $$ = $1; }
        | DO stmt WHILE '(' expr ')' ';'                { $$ = opr(DO, 2, $2, $5); }
        | FOR '(' assign ';' expr ';' assign ')' stmt { $$ = opr(FOR, 4, $3, $5, $7,
$9); }
...
assign:
          VARIABLE '=' expr     { $$ = opr('=', 2, id($1), $3); }
        ;
...
        | expr '%' expr         { $$ = opr('%', 2, $1, $3); }
        | expr '^' expr         { $$ = opr('^', 2, $1, $3); }
...
void yyerror(char *s) {
    fprintf(stdout, "line %d: %s near '%s'\n", yylineno, s, yytext);
}
```

- **Modulo and exponent.** `%` and `^` were added to the character class in lex, to the grammar and to all three back ends (`%` and `pow()` in the interpreter, new `mod` and `pow` instructions in the compiler, `[%]` and `[^]` in the graph). `%` has the same level as `*`. `^` is `%right` and is declared after `UMINUS`, so `2 ^ 3 ^ 2` is `2 ^ 9` and `-2 ^ 2` is `-(2 ^ 2)` like in maths.
- **for and do-while.** I added a new nonterminal `assign` so that the `for` header can have assignments without `;`. A `for` node has 4 children (init, condition, step, body) and a `do` node has 2 (body, condition). In the compiler `do-while` uses a new `jnz` (jump if not zero) instruction to jump back.
- **Multi-character names.** The lex rule for names is now `[a-zA-Z][a-zA-Z0-9_]*` and it is written after the keywords, so for `while` both rules match 5 characters and the keyword rule wins because it is first. `lookup()` in calc4.y searches the table `sym[]` of `{name, value}` entries and adds the name if it is new. The token value is the index in this table, so `sIndex` became an `int`.

- **Line numbers.** `%option yylineno` makes flex count lines. `yyerror()` prints `yylineno` and the text of the token where the error happened.

5. How I ran the examples

5.1 Makefile

I wrote one Makefile in `src/`. Each task is built in its own folder under `src/build/` because every yacc program makes its own `y.tab.c` and `y.tab.h` and they would overwrite each other. `make` builds everything, `make run` runs every input and saves the outputs, and `make clean` deletes all generated files.

Listing 15: *Part of src/Makefile (targets t3, t7 and bonus look the same as t2 and calc3, and the run target calls every program on every input file)*

```
CC = gcc
LEX = flex
YACC = bison -y -d
B = build
IN = ../inputs
OUT = ../outputs

all: t1 t2 t3 calc3 t7 bonus

t1:
    mkdir -p $(B)/t1
    for f in ex1 ex2 ex3 ex3mod ex4; do \
        $(LEX) -o $(B)/t1/$$f.c t1/$$f.l && \
        $(CC) $(B)/t1/$$f.c -o $(B)/t1/$$f -lfl || exit 1; \
    done

t2:
    mkdir -p $(B)/t2
    cd $(B)/t2 && $(YACC) ../../t2/calc1.y && $(LEX) ../../t2/calc1.l && \
        $(CC) y.tab.c lex.yy.c -o calc1


calc3:
    mkdir -p $(B)/calc3
    cd $(B)/calc3 && $(YACC) ../../calc3/calc3.y && $(LEX) ../../calc3/calc3.l && \
        $(CC) -I../../calc3 -c y.tab.c lex.yy.c && \
        $(CC) -I. -I../../calc3 y.tab.o lex.yy.o ../../calc3/calc3a.c -o calc3a && \
        $(CC) -I. -I../../calc3 y.tab.o lex.yy.o ../../calc3/calc3b.c -o calc3b && \
        $(CC) -I. -I../../calc3 y.tab.o lex.yy.o ../../calc3/calc3g.c -o calc3g

clean:
    rm -rf $(B)
```

5.2 Commands for each task

These are the commands that the Makefile runs (written by hand from inside each build folder):

Table 6: *Exact commands per task*

Task	Commands

T1	`flex -o ex1.c ex1.l`
	`gcc ex1.c -o ex1 -lfl`
	`./ex1 < inputs/t1_text.txt`
	`(the same for ex2, ex3, ex3mod, ex4)`
	`./ex2 inputs/t1_text.txt`

T2	`bison -y -d calc1.y`
	`flex calc1.l`
	`gcc y.tab.c lex.yy.c -o calc1`
	`./calc1 < inputs/t2_given1.txt`

T3	`bison -y -d calc2.y`
	`flex calc2.l`
	`gcc y.tab.c lex.yy.c -o calc2`
	`./calc2 < inputs/t3_given.txt`

T4 T5 T6	`bison -y -d calc3.y`

	flex calc3.l gcc -c y.tab.c lex.yy.c gcc y.tab.o lex.yy.o calc3a.c -o calc3a gcc y.tab.o lex.yy.o calc3b.c -o calc3b gcc y.tab.o lex.yy.o calc3g.c -o calc3g ./calc3a < inputs/t4_test.txt ./calc3b < inputs/t4_test.txt ./calc3g < inputs/t6_given.txt
T7	bison -y -d -v calc3np.y flex calc3.l gcc y.tab.c lex.yy.c calc3a.c -o calc3np bison -y -d -t calc3t.y flex calc3.l gcc y.tab.c lex.yy.c calc3a.c -o calc3t ./calc3t < inputs/t7_given.txt
Bonus	bison -y -d calc4.y flex calc4.l gcc -c y.tab.c lex.yy.c gcc y.tab.o lex.yy.o calc4a.c -o calc4a (same for calc4b and calc4g)

5.3 Problems I met and how I fixed them

1. **ex2 started counting from 2.** In the tutorial the counter in ex2.l is called `yylineno`. With flex this name already exists inside `lex.yy.c` and it starts at 1, so the first line was printed as line 2. I renamed my counter to `lineno` and the numbering started from 1.
2. **undefined reference to yywrap.** ex2, ex3 and ex4 do not define `yywrap()`. I fixed it by linking with `-lfl` like the project sheet says.
3. **calc1 gave a warning** `4 shift/reduce conflicts`. **It still builds. This is because calc1.y has no precedence lines. I left it like the tutorial and explain the effect in Section 6.2.**
4. **T7: bison error after removing the precedence lines.** When I deleted `%left GE LE EQ NE '>' '<'` bison said `symbol 'GE' is used, but is not defined as a token and has no rules`. That line was the only place where `GE LE EQ NE` were declared, so I replaced it with `%token GE LE EQ NE`.
5. **Wrong order of messages in saved outputs.** When I redirected the output to a file, `syntax error` (stderr) came before results that were printed earlier, because stdout is fully buffered when it goes to a file. I used `stdbuf -oL` in the Makefile so stdout is flushed after every line.
6. **y.tab.h overwritten between tasks.** At first I built everything in one folder and calc2 used the y.tab.h of calc3. Using a separate build folder for each task fixed it.

6. Sample runs

For every task I ran the inputs from the project sheet and at least three inputs of my own. Each listing shows the input file with `cat` and then the command and its output. The screenshots are from my VM.

6.1 T1: lex-only programs

Listing 16: *ex1 on a short text file*

$ cat inputs/t1_text.txt Hello world from my Ubuntu VM This is line two of the test file lex counts words lines and chars	$ ./ex1 < inputs/t1_text.txt Hello world from my Ubuntu VM This is line two of the test file lex counts words lines and chars

Correct: every character matches `.` or `\n` and is ECHOed, so the output is the same as the input.

Listing 17: *ex2 on the text file (left) and on my file where the last line has no newline (right)*

$ cat inputs/t1_text.txt Hello world from my Ubuntu VM This is line two of the test file	$ cat inputs/t1_lastline.txt first line second line

```
lex counts words lines and chars
$ ./ex2 inputs/t1_text.txt
   1    Hello world from my Ubuntu VM
   2    This is line two of the test file
   3    lex counts words lines and chars
```

```
last line has no newline
$ ./ex2 inputs/t1_lastline.txt
   1    first line
   2    second line
last line has no newline
```

Correct: the first file has three numbered lines. In my second file the last line does not end with `\n` so `^(.*)\n` cannot match it. No rule matches, so lex uses the default ECHO and the line is printed without a number.

Listing 18: *ex3 on my input with no match (left) and on the text file (right)*

```
$ cat inputs/t1_nomatch.txt
123 456 789
+-*/ !! ?? ==
$ ./ex3 < inputs/t1_nomatch.txt
123 456 789
+-*/ !! ?? ==
number of identifiers = 0
```

```
$ cat inputs/t1_text.txt
Hello world from my Ubuntu VM
This is line two of the test file
lex counts words lines and chars
$ ./ex3 < inputs/t1_text.txt



number of identifiers = 20
```

Correct: in the first file there is no letter so the identifier rule never matches. All characters are ECHOed and the count is 0. In the text file there are 20 words and all of them are identifiers. Only the spaces and newlines are not matched, so they are ECHOed and the output looks empty.

Listing 19: *ex3 with the original rule (left) and ex3mod with my changed rule (right) on my C-like code*

```
$ cat inputs/t1_code.txt
int my_var = count2 + 10;
float _tmp = my_var * 3;
$ ./ex3 < inputs/t1_code.txt
 _ =  + 10;
 _ = _ * 3;
number of identifiers = 8
```

```
$ cat inputs/t1_code.txt
int my_var = count2 + 10;
float _tmp = my_var * 3;
$ ./ex3mod < inputs/t1_code.txt
  =  + 10;
  =  * 3;
number of identifiers = 6
```

Effect of my change: with the original rule `my_var` counts as `my` and `var` and the _ is ECHOed, so there are 8 identifiers (int, my, var, count2, float, tmp, my, var). With _ added to `letter` there are 6 (int, my_var, count2, float, _tmp, my_var) and no _ is left in the output.

Listing 20: *ex4 on three files (output is characters, words, lines)*

```
$ cat inputs/t1_text.txt
Hello world from my Ubuntu VM
This is line two of the test file
lex counts words lines and chars
$ ./ex4 < inputs/t1_text.txt
97    20    3
```

```
$ cat inputs/t1_nomatch.txt
123 456 789
+-*/ !! ?? ==
$ ./ex4 < inputs/t1_nomatch.txt
26    7    2
```

```
$ cat inputs/t1_lastline.txt
first line
second line
last line has no newline
$ ./ex4 < inputs/t1_lastline.txt
47    9    2
```

Correct: I checked all three with `wc` and it gives the same numbers in the order lines, words, chars. For example the text file has 3 lines, 20 words and 97 characters. The last file has only 2 lines because the last line has no `\n`.

[Paste screenshot here: terminal with the T1 runs]

Figure 5: *Terminal running the T1 lex programs*

6.2 T2: calc1 with + and -

Listing 21: *calc1 on the three inputs from the project sheet*

```
$ cat inputs/t2_given1.txt
1 + 2
$ ./calc1 <
inputs/t2_given1.txt
3
```

```
$ cat inputs/t2_given2.txt
10 - 4 + 3
$ ./calc1 <
inputs/t2_given2.txt
3
```

```
$ cat inputs/t2_given3.txt
5 +
$ ./calc1 <
inputs/t2_given3.txt
syntax error
```

`1 + 2` gives 3 which is correct. `10 - 4 + 3` gives **3** and not 9. This is because of the 4 shift/reduce conflicts: after `10 - 4` the parser sees `+` and bison chose shift, so it computed `10 - (4 + 3)`. So the program does what the grammar says, but the grammar makes `+` and `-` right associative. This is exactly why `%left` is needed (see Section 8).

`5 +` is the error case: after `+` the parser needs an `expr` but gets `'\n'`, so there is no rule and it calls `yyerror("syntax error")`.

Listing 22: *calc1 on my three inputs (the third is an error case)*

```
$ cat inputs/t2_own1.txt
7
20 + 30 + 40
100 - 1 - 1
$ ./calc1 <
inputs/t2_own1.txt
7
90
100
```

```
$ cat inputs/t2_own2.txt
8 - 2 - 1
1 - 1 + 1
$ ./calc1 <
inputs/t2_own2.txt
7
-1
```

```
$ cat inputs/t2_own3.txt
3 * 4
$ ./calc1 <
inputs/t2_own3.txt
invalid character
syntax error
```

My inputs show the same right associativity: `100 - 1 - 1` is `100 - (1 - 1) = 100`, `8 - 2 - 1` is `8 - (2 - 1) = 7` and `1 - 1 + 1` is `1 - (1 + 1) = -1`. Additions like `20 + 30 + 40 = 90` are still right because the order does not matter for `+`. In the last input `*` is not in calc1, so lex prints `invalid character` and skips it. Then the parser gets `3 4` which is a syntax error.

[Paste screenshot here: terminal with the T2 runs]

Figure 6: *Terminal running calc1 (T2)*

6.3 T3: calc2 with variables, * / and brackets

Listing 23: *calc2 on the inputs from the project sheet*

```
$ cat inputs/t3_given.txt
x = 3 + 4 * 2
x
(x + 1) * 2
y
```

```
$ ./calc2 < inputs/t3_given.txt
11
24
0
```

Correct: `x = 3 + 4 * 2` stores 11 because `*` is done first, and an assignment prints nothing. `x` prints 11, `(x + 1) * 2` is 24. `y` was never assigned, it prints 0 because `sym` is a global array and starts with zeros.

Listing 24: *calc2 on my three inputs (precedence, variables, syntax error)*

```
$ cat inputs/t3_own1.txt
2 + 3 * 4
(2 + 3) * 4
20 / 2 / 5
$ ./calc2 <
inputs/t3_own1.txt
14
20
2
```

```
$ cat inputs/t3_own2.txt
a = 10
b = a / 3
b * 3 + a - 2
$ ./calc2 <
inputs/t3_own2.txt
17
```

```
$ cat inputs/t3_own3.txt
z = 5
z + * 2
$ ./calc2 <
inputs/t3_own3.txt
syntax error
```

Correct: `2 + 3 * 4 = 14` and with brackets `20`. `20 / 2 / 5` is `(20 / 2) / 5 = 2` because of `%left` (right associative would compute `20 / (2 / 5) = 20 / 0` and crash). In input 2, `b = 10 / 3 = 3` (integer division) and `3 * 3 + 10 - 2 = 17`. In input 3 `z + * 2` has two operators next to each other, so it is a syntax error and the program stops.

[Paste screenshot here: terminal with the T3 runs]

Figure 7: *Terminal running calc2 (T3)*

6.4 T4: calc3 interpreter

Listing 25: *calc3a on the test program from the project sheet*

```
$ cat inputs/t4_test.txt
x = 0;
while (x < 3) {
    print x;
    x = x + 1;
}
if (x == 3) print 100; else print 200;
```

```
$ ./calc3a < inputs/t4_test.txt
0
1
2
100
```

Correct: this is exactly the expected output 0, 1, 2, 100. The loop prints x while `x < 3` and after the loop `x == 3` is true so it prints 100.

Listing 26: *My input 1: factorial of 5 with while*

```
$ cat inputs/t4_own1.txt
n = 5;
f = 1;
while (n > 1) {
    f = f * n;
    n = n - 1;
}
print f;
```

```
$ ./calc3a < inputs/t4_own1.txt
120
```

Listing 27: *calc3a on my input 2 (dangling else, unary minus) and input 3 (syntax error)*

```
$ cat inputs/t4_own2.txt
a = 1;
b = 0;
if (a) if (b) print 1; else print 2;
if (b) print 3;
print -3 * 2 + 10;
print 7 / 2;
print a != b;
$ ./calc3a < inputs/t4_own2.txt
2
4
3
1
```

```
$ cat inputs/t4_own3.txt
x = 4;
print x +;
print 99;
$ ./calc3a < inputs/t4_own3.txt
syntax error
```

Correct: 5! = 120. In input 2 the `else` belongs to the inner `if (b)`, so it prints 2 (if it belonged to the outer if nothing would be printed). `if (b) print 3;` prints nothing because b is 0. `-3 * 2 + 10 = 4`, `7 / 2 = 3` and `a != b` is 1. In input 3 `x +;` is a syntax error, so the parser stops and `print 99` is never run. The first statement was already run, which shows that every statement is executed as soon as it is parsed.

[Paste screenshot here: terminal with the T4 runs]

Figure 8: *Terminal running calc3a (T4)*

6.5 T5: calc3 compiler

Listing 28: *calc3b on the test program*

```
$ cat inputs/t4_test.txt
x = 0;
```

```
$ ./calc3b <
inputs/t4_test.txt
```

```
    jmp    L000
L001:
```

```
while (x < 3) {
    print x;
    x = x + 1;
}
if (x == 3) print 100; else
print 200;
```

```
    push    0
    pop    x
L000:
    push    x
    push    3
    compLT
    jz    L001
    push    x
    print
    push    x
    push    1
    add
    pop    x
```

```
    push    x
    push    3
    compEQ
    jz    L002
    push    100
    print
    jmp    L003
L002:
    push    200
    print
L003:
```

Explanation of the generated code. The code is for a stack machine. `push 0` and `pop x` is `x = 0`. `L000` is the top of the while loop: it pushes x and 3, `compLT` replaces them with 1 or 0 and `jz L001` leaves the loop when the result is 0. The body prints x and does `x = x + 1` with `push x, push 1, add, pop x`, and `jmp L000` goes back. After the loop the if-else compares `x == 3` with `compEQ`, `jz L002` jumps to the else part, and the then part ends with `jmp L003` to skip the else part. If I run this by hand on a stack it prints 0, 1, 2, 100, the same as the interpreter, so the code is correct.

Listing 29: *calc3b on my input 2 (nested if)*

```
$ cat inputs/t4_own2.txt
a = 1;
b = 0;
if (a) if (b) print 1; else
print 2;
if (b) print 3;
print -3 * 2 + 10;
print 7 / 2;
print a != b;
```

```
$ ./calc3b <
inputs/t4_own2.txt
    push    1
    pop    a
    push    0
    pop    b
    push    a
    jz    L000
    push    b
    jz    L001
    push    1
    print
    jmp    L002
L001:
    push    2
    print
L002:
L000:
    push    b
    jz    L003
```

```
    push    3
    print
L003:
    push    3
    neg
    push    2
    mul
    push    10
    add
    print
    push    7
    push    2
    div
    print
    push    a
    push    b
    compNE
    print
```

The output for my input 1 (factorial) is in `outputs/t5_own1.txt` and it has one loop like the test program. For the nested if there are two labels for the inner if-else (`L001`, `L002`) inside the outer if which only has one label (`L000`) because it has no else. Unary minus becomes `neg` after its operand.

Listing 30: *calc3b on my input 3 (error)*

```
$ cat inputs/t4_own3.txt
x = 4;
print x +;
print 99;
```

```
$ ./calc3b < inputs/t4_own3.txt
    push    4
    pop    x
syntax error
```

The first statement is compiled and then the error stops the parser.

[Paste screenshot here: terminal with the T5 runs]

Figure 9: *Terminal running calc3b (T5)*

6.6 T6: syntax tree drawing

Listing 31: *calc3g on x = 2 * (3 + 4);*

```
$ cat inputs/t6_given.txt
x = 2 * (3 + 4);
```

```
$ ./calc3g < inputs/t6_given.txt


Graph 0:

          [=]
           |
    |------|
    |      |
  id(X)   [*]
           |
      |----|
      |    |
    c(2)  [+]
           |
        |--|
        |  |
      c(3) c(4)
```

Correct: the root is the assignment `[=]` with the variable on the left and `[*]` on the right. The `+` is lower than the `*` because the brackets made it a sub-expression. There is no node for the brackets. Variables are printed in capital letters (`id(X)`) by the drawing code.

Listing 32: *calc3g on my input 1*

```
$ cat inputs/t6_own1.txt
print 1 + 2 * 3;
```

```
$ ./calc3g < inputs/t6_own1.txt


Graph 0:

     print
       |
       |
       |
      [+]
       |
  |----|
  |    |
c(1)  [*]
       |
    |--|
    |  |
  c(2) c(3)
```

Listing 33: *calc3g on my input 2 (if-else and unary minus) and input 3 (while)*

```
$ cat inputs/t6_own2.txt
if (x > 0) print x; else print -x;
$ ./calc3g < inputs/t6_own2.txt


Graph 0:

           if
           |
     |-----|-----|
     |           |
    [>]      print print
     |         |     |
   |-|-|       |     |
   |   |       |     |
 id(X) c(0) id(X)  [_]
                     |
                     |
                     |
                   id(X)
```

```
$ cat inputs/t6_own3.txt
while (i < 3) i = i + 1;
$ ./calc3g < inputs/t6_own3.txt


Graph 0:

            while
              |
     |--------|--------|
     |                 |
    [<]               [=]
     |                 |
   |-|-|       |-------|-------|
   |   |       |               |
 id(I) c(3) id(I)             [+]
                               |
                            |--|
                            |  |
                          id(I) c(1)
```

In my input 1 * is under + because it has higher precedence. The if-else node has 3 children (condition, then, else) and unary minus is drawn as [_]. The while node has 2 children: the condition and the body.

[Paste screenshot here: terminal with the T6 runs]

Figure 10: *Terminal running calc3g (T6)*

6.7 Bonus runs

Listing 34: *calc4a (interpreter) on my bonus test program*

```
$ cat inputs/bonus_test.txt
total = 0;
for (i = 1; i <= 5; i = i + 1) total =
total + i;
print total;
count = 3;
do {
    print count;
    count = count - 1;
} while (count > 0);
print 17 % 5;
print 2 ^ 3 ^ 2;
print -2 ^ 2;
print 10 - 4 - 3;
```

```
$ ./calc4a < inputs/bonus_test.txt
15
3
2
1
2
512
-4
3
```

Correct: the for loop adds 1 to 5 so total is 15. The do-while prints 3, 2, 1. `17 % 5 = 2`. `2 ^ 3 ^ 2 = 2 ^ 9 = 512` because ^ is right associative. `-2 ^ 2 = -4` because ^ binds stronger than unary minus. `10 - 4 - 3 = 3` shows that - is still left associative.

The compiler output for this program is in `outputs/bonus_compile.txt`. The for loop becomes a loop with `jz` at the top like the while loop, and the do-while ends with `jnz L002` which jumps back while `count > 0`. The new operators produce `mod` and `pow`, and `2 ^ 3 ^ 2` gives `push 2 push 3 push 2 pow pow`, which is right associative.

Listing 35: *calc4a error message with the line number*

```
$ cat inputs/bonus_err.txt
value = 10;
print value;
print value * ;
```

```
$ ./calc4a < inputs/bonus_err.txt
10
line 3: syntax error near ';'
```

Listing 36: *calc4g with a multi-character name, ^ and %*

```
$ cat inputs/bonus_graph.txt
big_num = 2 ^ 10 % 7;
```

```
$ ./calc4g < inputs/bonus_graph.txt


Graph 0:

             [=]
              |
      |-------------|
      |             |
 id(big_num)       [%]
                    |
                 |-------|
                 |       |
               [^]     c(7)
                |
             |-----|
             |     |
           c(2) c(10)
```

The error is on line 3 and the message says so. In the tree `^` is below `%` because `^` has higher precedence, and the variable name `big_num` is printed in full.

[Paste screenshot here: terminal with the bonus runs]

Figure 11: *Terminal running the bonus calculator*

7. Experiments (T7)

7.1 Removing the precedence lines

I copied calc3.y to `src/t7/calc3np.y` and removed the three `%left` lines (I had to keep `%token GE LE EQ NE`, see Section 5.3). I kept `%nonassoc IFX/ELSE` and `%nonassoc UMINUS`. Then I built it with `bison -y -d -v` and bison said:

```
calc3np.y: warning: 110 shift/reduce conflicts [-Wconflicts-sr]
```

The beginning of `y.output` lists where the conflicts are:

Listing 37: *Conflict summary at the top of y.output*

```
State 23 conflicts: 10 shift/reduce
State 42 conflicts: 10 shift/reduce
State 43 conflicts: 10 shift/reduce
State 44 conflicts: 10 shift/reduce
State 45 conflicts: 10 shift/reduce
State 46 conflicts: 10 shift/reduce
State 47 conflicts: 10 shift/reduce
State 48 conflicts: 10 shift/reduce
State 49 conflicts: 10 shift/reduce
State 50 conflicts: 10 shift/reduce
State 51 conflicts: 10 shift/reduce
```

There are 11 states with 10 conflicts each. I checked the states in y.output: states 42 to 51 are the ten binary rules right before a reduce (`expr GE expr .`, `expr '+' expr .`, `expr '*' expr .` and so on) and state 23 is the unary minus rule `'-' expr .`. In each of them there are 10 operator tokens that could come next, so 11 x 10 = 110. Here is state 48:

Listing 38: *State 48 of y.output (expr '*' expr .), left: items and shifts, right: the reduces bison did not use*

```
State 48

   17 expr: expr . '+' expr
   18     | expr . '-' expr
   19     | expr . '*' expr
   19     | expr '*' expr .
   20     | expr . '/' expr
   21     | expr . '<' expr
   22     | expr . '>' expr
   23     | expr . GE expr
   24     | expr . LE expr
   25     | expr . NE expr
   26     | expr . EQ expr

    GE   shift, and go to state 24
    LE   shift, and go to state 25
    EQ   shift, and go to state 26
    NE   shift, and go to state 27
    '-'  shift, and go to state 29
    '+'  shift, and go to state 30
    '*'  shift, and go to state 31
    '/'  shift, and go to state 32
    '<'  shift, and go to state 33
    '>'  shift, and go to state 34
```

```
    GE        [reduce using rule 19 (expr)]
    LE        [reduce using rule 19 (expr)]
    EQ        [reduce using rule 19 (expr)]
    NE        [reduce using rule 19 (expr)]
    '-'       [reduce using rule 19 (expr)]
    '+'       [reduce using rule 19 (expr)]
    '*'       [reduce using rule 19 (expr)]
    '/'       [reduce using rule 19 (expr)]
    '<'       [reduce using rule 19 (expr)]
    '>'       [reduce using rule 19 (expr)]
    $default  reduce using rule 19 (expr)
```

The dot shows where the parser is. In state 48 the parser has `expr '*' expr` on the stack. If the next token is `'+'` it can shift it (the dot moves in `expr . '+' expr`) or reduce rule 19. bison has no precedence to decide, so it always picks shift and puts the reduce in brackets, which means it is never used. The `$default reduce` line is used for all other tokens like `;` or `)`.

Listing 39: *Without precedence on 1 + 2 * 3 (left), without precedence on my inputs (middle), original calc3a on my inputs (right)*

```
$ cat inputs/t7_given.txt
print 1 + 2 * 3;
$ ./calc3np <
inputs/t7_given.txt
7
```

```
$ cat inputs/t7_own.txt
print 2 * 3 + 1;
print 10 - 4 - 3;
print 1 < 2 + 3;
print -2 + 5;
$ ./calc3np < inputs/t7_own.txt
8
9
1
-7
```

```
$ cat inputs/t7_own.txt
print 2 * 3 + 1;
print 10 - 4 - 3;
print 1 < 2 + 3;
print -2 + 5;
$ ./calc3a < inputs/t7_own.txt
7
3
1
3
```

Without precedence everything becomes right associative with the same level. `1 + 2 * 3` is still 7 by luck because shift gives `1 + (2 * 3)`. But `2 * 3 + 1` becomes `2 * (3 + 1) = 8`, `10 - 4 - 3` becomes `10 - (4 - 3) = 9` and `-2 + 5` becomes `-(2 + 5) = -7`. The last one is interesting: the rule has precedence `UMINUS` but `'+'` has no precedence now, and bison only uses precedence when both the rule and the token have one, so it is a conflict again. With the original grammar the answers are 7, 3, 1 and 3, which are correct.

7.2 Parser trace with yydebug

For the trace I made `src/t7/calc3t.y`, which is calc3.y with one more line `yydebug = 1;` in `main()`. I built it with `bison -t` which defines `YYDEBUG` so the trace code is compiled in. The trace is printed to stderr. This uses the normal grammar with precedence.

Listing 40: *Parser trace for print 1 + 2 * 3; (read the left column first, then the right column)*

```
$ ./calc3t < inputs/t7_given.txt
Starting parse
Entering state 0
Stack now 0
Reducing stack by rule 3 (line 45):
-> $$ = nterm function ()
Entering state 2
Stack now 0 2
Reading a token
Next token is token PRINT ()
Shifting token PRINT ()
Entering state 8
Stack now 0 2 8
Reading a token
Next token is token INTEGER ()
Shifting token INTEGER ()
Entering state 4
Stack now 0 2 8 4
Reducing stack by rule 14 (line 65):
   $1 = token INTEGER ()
-> $$ = nterm expr ()
Entering state 19
Stack now 0 2 8 19
Reading a token
Next token is token '+' ()
Shifting token '+' ()
Entering state 30
Stack now 0 2 8 19 30
Reading a token
Next token is token INTEGER ()
Shifting token INTEGER ()
Entering state 4
Stack now 0 2 8 19 30 4
Reducing stack by rule 14 (line 65):
   $1 = token INTEGER ()
-> $$ = nterm expr ()
Entering state 48
Stack now 0 2 8 19 30 48
Reading a token
Next token is token '*' ()
```

```
Reducing stack by rule 14 (line 65):
   $1 = token INTEGER ()
-> $$ = nterm expr ()
Entering state 50
Stack now 0 2 8 19 30 48 32 50
Reducing stack by rule 19 (line 70):
   $1 = nterm expr ()
   $2 = token '*' ()
   $3 = nterm expr ()
-> $$ = nterm expr ()
Entering state 48
Stack now 0 2 8 19 30 48
Reading a token
Next token is token ';' ()
Reducing stack by rule 17 (line 68):
   $1 = nterm expr ()
   $2 = token '+' ()
   $3 = nterm expr ()
-> $$ = nterm expr ()
Entering state 19
Stack now 0 2 8 19
Next token is token ';' ()
Shifting token ';' ()
Entering state 38
Stack now 0 2 8 19 38
Reducing stack by rule 6 (line 51):
   $1 = token PRINT ()
   $2 = nterm expr ()
   $3 = token ';' ()
-> $$ = nterm stmt ()
Entering state 13
Stack now 0 2 13
Reducing stack by rule 2 (line 44):
   $1 = nterm function ()
   $2 = nterm stmt ()
7
-> $$ = nterm function ()
Entering state 2
Stack now 0 2
Reading a token
```

```
Shifting token '*' ()
Entering state 32
Stack now 0 2 8 19 30 48 32
Reading a token
Next token is token INTEGER ()
Shifting token INTEGER ()
Entering state 4
Stack now 0 2 8 19 30 48 32 4
```

```
Now at end of input.
Reducing stack by rule 1 (line 40):
   $1 = nterm function ()
```

Explanation. First the parser reduces the empty rule to `function` (rule 3). Then it shifts `PRINT` and `INTEGER(1)` and reduces the integer to `expr` (rule 14). It shifts `'+'` and `INTEGER(2)` and reduces again, so the stack has `expr '+' expr` (state 48 in this build). Now the next token is `'*'`. This is the important point: the parser does **not** reduce the addition, it shifts `'*'` because `'*'` has higher precedence than the `'+'` rule. It reads `3`, reduces it, and then reduces `expr '*' expr` (rule 19) first. Then it is back in state 48 with lookahead `';'`, and now it reduces `expr '+' expr` (rule 17). Then it shifts `;`, reduces `PRINT expr ';'` to `stmt` (rule 6) and `function stmt` to `function` (rule 2). At this reduce the action calls `ex()` and the interpreter prints 7, which can be seen in the trace. At the end of input it reduces rule 1 and the action calls `exit(0)`.

[Paste screenshot here: terminal with bison -v output and the yydebug trace]

Figure 12: *Terminal with the bison conflict warning and the parser trace (T7)*

8. Answers to the key questions

What is a token, and what is the difference between the token type and its value?

A token is the smallest unit that has a meaning for the parser, for example a number, a variable name, a keyword or an operator. The **type** is what kind of token it is. It is the integer that `yylex()` returns, like `INTEGER` (258) or `'+'` (43). The parser only needs the type to decide what to do with the grammar. The **value** is the extra information, like which number it is or which variable. Lex puts it into `yylval` before returning, and yacc pushes it on the value stack so the actions can use it as `$1`, `$2`. For example `42` and `7` have the same type `INTEGER` but different values.

Why does yacc need precedence declarations such as %left '+' '-'?

A grammar like `expr: expr '+' expr | expr '*' expr` is ambiguous. `1 + 2 * 3` can be parsed in two ways and `10 - 4 - 3` too. The LALR parser finds this as shift/reduce conflicts. `%left` and `%right` tell yacc how to choose: the line order gives the precedence (later is higher) and left or right gives the associativity when the precedence is the same. Without them bison always shifts, which gave wrong answers in my runs, like `10 - 4 + 3 = 3` in calc1 and `2 * 3 + 1 = 8` in T7. The other way is to rewrite the grammar with more nonterminals (expr, term, factor), but precedence lines are shorter and easier to read.

What is a shift/reduce conflict? Show one that occurs if you remove the precedence rules.

It happens when the parser is in a state where it could both shift the next token and reduce a rule, and the grammar does not say which is right. In state 48 from Section 7.1 the stack has `expr '*' expr` and the next token is `'+'`. Reducing first means `(2 * 3) + 1` and shifting first means `2 * (3 + 1)`. Both are allowed by the grammar. bison reports `110 shift/reduce conflicts`, chooses shift and we got 8 instead of 7. A reduce/reduce conflict is when two different rules can be reduced at the same point; this did not happen in my grammars.

How does the calculator resolve the dangling-else ambiguity?

In `if (a) if (b) s1; else s2;` the `else` could belong to either `if`. When the parser has `IF '(' expr ')' stmt` on the stack and sees `ELSE`, it can reduce the short if or shift the else. calc3.y declares `%nonassoc IFX` and then `%nonassoc ELSE`, so ELSE has higher precedence. The short if rule gets the precedence of IFX with `%prec IFX`. Because the token ELSE is higher than the rule, bison shifts, so the else belongs to the closest if. This is the same as C. My input 2 in T4 shows it: the program printed 2, which means the else was attached to `if (b)`. Without these lines bison would still shift by default but it would show a warning.

9. Conclusion

In this project I learned how a real compiler front end is made from two generated parts. Before, lex and yacc looked like magic to me, but after running each example I understood that `yylex()` is just a function that returns numbers and `yylval`, and `yyparse()` is a stack machine that shifts and reduces. The most useful part for me was the calculator, because the same parser and syntax tree can be interpreted, compiled to stack code or drawn just by linking a different `ex()` function. It showed me why compilers build a tree and do not do everything in the parser.

The difficult parts were the conflicts and reading `y.output`. At first I did not understand why calc1 printed 3 for `10 - 4 + 3`, and I only understood it after I read the states in y.output and the parser trace. Also some small things took time, like the `yylineno` name in ex2, the undefined `GE` after removing the `%left` lines and the stdout buffering problem. For the bonus the hardest part was the for loop, because the `for` header needs assignments without a semicolon, so I had to add a new nonterminal.

Declaration of AI tool use

[Edit this paragraph so it is true for you.] I used an AI assistant (Claude by Anthropic) during this project to help me write and organise the source files, the Makefile and parts of this report. I read and checked every explanation, built every program and produced the outputs and screenshots on my own VM.

10. References

7. T. Niemann, "LEX & YACC TUTORIAL", ePaperPress. https://www.epaperpress.com/lexandyacc/ (tutorial PDF and calculator source code; calc3g.c graph code by Frank Thomas Braun).
8. The Flex Project, "Lexical Analysis With Flex" manual, version 2.6.4. https://westes.github.io/flex/manual/
9. Free Software Foundation, "GNU Bison - The Yacc-compatible Parser Generator" manual, version 3.8. https://www.gnu.org/software/bison/manual/
10. Istanbul Medipol University, "Mini Project: Building a Compiler with Lex & Yacc", Fundamentals of Programming Languages, Fall 2026 project sheet.